

Evaluating Graph-Based and Traditional Machine Learning Models for Airbnb Price Prediction

Stanford CS229 Project

Ethan Cohen
ICME
Stanford University
ethancoh@stanford.edu

Haotian (Owen) Cui
ICME
Stanford University
owencui@stanford.edu

Matthieu Hautsch
Civil & Environmental Engineering
Stanford University
matthaut@stanford.edu

1 Introduction

This project was inspired by three international students who share a passion for traveling. While exchanging travel stories, we noticed that Airbnb consistently emerged as the most popular accommodation choice. This observation raised a natural question: *what factors allow Airbnb to fairly and accurately determine a listing's price?* Accurate pricing is important for hosts seeking fair revenue, for guests hoping to avoid overpaying, and for platforms that rely on consistent valuation across diverse neighborhoods.

To explore this question, we train machine learning models to predict Airbnb listing prices in the San Francisco area. Our inputs combine listing-level attributes (*description text, neighborhood, accommodates, room type, bathrooms, bedrooms, beds, amenities*), spatial information (*latitude, longitude*), and review-based indicators (*number of reviews, review scores rating, host is superhost, availability within 30 days, availability within 365 days*). After preprocessing these inputs, we benchmark a broad set of models, including *Linear Regression, LASSO, Ridge, Random Forests, XGBoost, K-Nearest Neighbors, a simple neural network, and an inductive GraphSAGE (GNN) model*, to predict the *log-transformed listing price*.

Our motivation for testing multiple model classes is to assess how well standard tabular methods perform under realistic spatial generalization, and to determine whether a graph neural network can more effectively exploit neighborhood structure than non-spatial baselines.

2 Related Work

Prior studies on Airbnb pricing span three main methodological directions: classical regression models, machine learning approaches, and, more recently, graph-based methods.

Classical regression: Early studies model short-term rental prices using hedonic or linear regression. Wang and Nicolau (2017) analyze over 180,000 listings across 33 cities using OLS and quantile regression, identifying host characteristics, property attributes, amenities, rental rules, and online reviews as key price determinants; these insights informed our own feature design. Similarly, Yang et al. (2016) show that hotel prices are shaped by accessibility and quality signals, underscoring that location frictions and reputation effects are central drivers of accommodation pricing. Classical regression offers interpretability but cannot model nonlinear interactions or neighborhood structure.

Machine learning models: More recent work applies ML techniques using structured, textual, or visual features. For example, Tang and Sangani (2015) train SVMs on sentiment, text, and image-derived descriptors from the Inside Airbnb dataset to classify listings into low- or high-price buckets, demonstrating that descriptive text carries strong predictive signal. However, their formulation is a coarse classification task rather than a continuous regression problem, and it does not account for spatial dependencies. These models improve flexibility but ignore relational structure across listings.

Graph neural networks for price prediction: Closest to our approach, Kanakaris and Karacapilidis (2023) construct a graph of geographically and textually similar Airbnb listings and apply a GCN with Doc2Vec embeddings, reporting improvements in R^2 and MSE over classical baselines. Their method, however, is *transductive*: the entire global graph is fixed and test nodes are already present (though unlabeled) during training. In contrast, our GraphSAGE model follows the *inductive* paradigm of Hamilton et al. (2017), learning neighborhood aggregation functions that generalize to completely unseen regions. This allows genuine spatial generalization while integrating tabular features, spatial structure, and text-derived embeddings.

3 Dataset and Features

Our dataset is obtained from the Inside Airbnb open data project ins, which provides publicly available snapshots of Airbnb listings across major cities. We use the San Francisco listings dataset for the purpose of our project.

Data Preprocessing: Several preprocessing steps were applied. We first restricted the data to short-term stays, then cleaned prices by removing missing values and capping extreme outliers. We applied a logarithmic transformation to the target variable, $\log_price$, to reduce right skewness and stabilize variance. Text descriptions were cleaned by removing non-informative or noisy phrases. We also engineered additional numerical features to capture listing comfort, including $bedrooms_per_guest$, $bathrooms_per_guest$, $beds_per_guest$, and an $amenity_count$ feature derived from the raw amenities list. After all cleaning and filtering, we retained a total of 5,182 listings.

Text Embeddings: To incorporate semantic information from listing descriptions, we embedded each cleaned description using the Sentence-BERT (SBERT) model `all-MiniLM-L6-v2` Reimers and Gurevych (2019). SBERT is a transformer-based model fine-tuned so that semantically similar sentences are mapped to nearby points in embedding space, making cosine distance a meaningful proxy for textual similarity. The `all-MiniLM-L6-v2` variant is a lightweight SBERT model that efficiently produces 384-dimensional sentence embeddings. Using the `sentence_transformers` library, we obtained for each listing a vector $embedding_sbert \in \mathbb{R}^{384}$, which we then used as additional features alongside the tabular attributes.

Dataset Versions and Splits:

Random Split Dataset D1 (No Text Embeddings): To establish baseline performance and highlight the effect of spatial leakage, we created an 80/20 random train-test split consisting of 4,145 training listings and 1,037 test listings. This version excludes SBERT embeddings and is used for classical tabular models and initial comparison. We do *not* train the GNN on this dataset since random spatial mixing violates the inductive testing scenario.

Geo-Aware Split Dataset D2 (With Embeddings and Clusters): To evaluate spatial generalization and train our inductive GraphSAGE model, we constructed a second dataset containing all engineered features, SBERT embeddings, and an additional `geo_cluster` feature. The clusters were obtained by applying k -means ($k = 20$) to the geographic coordinates (latitude and longitude). We then held out 5 entire clusters as the test set, yielding 4,402 training listings (85%) and 780 test listings (15%). Because the model never sees the test clusters during training, this split provides a realistic assessment of performance on unseen neighborhoods (Figure 1).

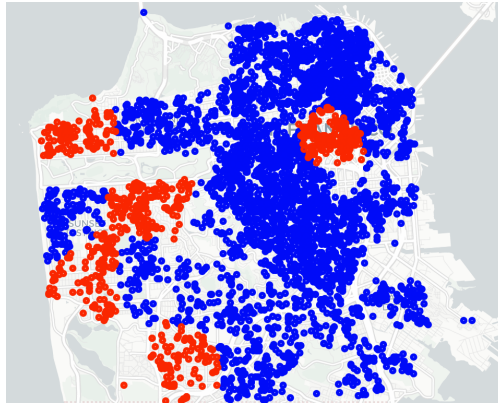


Figure 1: Geographical Train (Blue) - Test (Red) Split of San Francisco Airbnb Listings

4 Methods

In this section, we describe the learning algorithms used to predict the log-transformed Airbnb price. Let $x \in \mathbb{R}^d$ denote the feature vector of a listing (after preprocessing and encoding) and let $y = \log(\text{price})$ be the target. We compare linear, distance-based, tree-based, and neural models.

K-Nearest Neighbors (KNN, Baseline): Our baseline model is k -Nearest Neighbors, which predicts a listing’s log-price from the local neighborhood of similar listings. In practice, we use a pipeline with feature standardization and, in some configurations, PCA-based dimensionality reduction before applying KNN. Given a query listing with features x , KNN identifies the set $\mathcal{N}_k(x)$ of its k nearest neighbors in the transformed feature space and predicts

$$\hat{y}(x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} y_i,$$

where $y_i = \log(\text{price}_i)$ are the targets of the neighbors. This non-parametric approach directly operationalizes the intuition behind Airbnb pricing: listings with similar location, capacity, amenities, and textual descriptions should have comparable prices. KNN is a natural baseline for Airbnb pricing because it mimics how prices are set in practice: by looking at nearby “comparable” listings. It is simple, and has only a few intuitive hyperparameters. This makes it an appropriate reference point: more sophisticated models (e.g., tree ensembles or GNNs) should at least match or exceed the performance of this locality-based predictor.

Linear Models (OLS, LASSO, Ridge): Linear models trained with the Least Mean Squares (LMS) criterion assume $y_i \approx x_i^\top \beta + \varepsilon_i$, and in the unregularized case estimate β by minimizing $\frac{1}{n} \sum_{i=1}^n (y_i - x_i^\top \beta)^2$. Because the target is log-price, this objective corresponds to minimizing squared errors on log-prices. To improve generalization in high-dimensional feature space, we use LASSO and Ridge, which solve

$$\min_{\beta} \sum_{i=1}^n (y_i - x_i^\top \beta)^2 + \lambda \|\beta\|_1 \quad (\text{LASSO}), \quad \min_{\beta} \sum_{i=1}^n (y_i - x_i^\top \beta)^2 + \lambda \|\beta\|_2^2 \quad (\text{Ridge}).$$

LASSO encourage sparsity and implicit feature selection, while Ridge shrinks coefficients smoothly toward zero. These linear models capture global trends in Airbnb prices (e.g., how price scales with capacity or location indicators).

Random Forest: Random Forests build on regression trees, which partition the feature space into axis-aligned regions and predict each listing’s log-price as the average target value in the corresponding leaf, allowing them to capture nonlinearities and threshold effects (e.g., price jumps above a certain number of guests). A Random Forest reduces the variance of individual trees by averaging an ensemble of trees trained on bootstrap samples (bagging) and random subsets of features at each split. Given B trees $\{T_b\}_{b=1}^B$, the prediction for a listing is $\hat{y}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$. The injected randomness decorrelates the trees, so averaging reduces variance while still capturing complex nonlinear relations between features and log-price.

XGBoost (Gradient-Boosted Trees): XGBoost is a gradient-boosted tree model that sequentially adds shallow regression trees to correct the residual errors of the current ensemble. Starting from an initial prediction (e.g., the mean log-price), each new tree is fit to the negative gradients of the loss, and its contribution is scaled by a learning rate. This boosting procedure mainly targets bias reduction and allows the model to capture nonlinear relationships with relatively shallow trees, with additional regularization terms to control model capacity.

Neural Network (Multilayer Perceptron): A fully connected feedforward neural network (multilayer perceptron) learns a nonlinear mapping from features to log-price. The network consists of several hidden layers with ReLU activations, batch normalization, dropout, and a final linear output layer, and its parameters are learned by minimizing the mean squared error loss on the log-price using stochastic gradient-based optimization (e.g., Adam). This architecture matches our feature representation, which combines dense text-derived embeddings with structured inputs.

Graph Neural Network (GraphSAGE): A graph neural network allows us to exploit the relational structure between listings. We construct a graph $G = (V, E)$ where each node $i \in V$ corresponds to an Airbnb listing, with initial node features $x_i \in \mathbb{R}^{d=15}$ given by the same preprocessed feature vector as in the previous models.

Edges connect listings that are similar according to two criteria. First, we build a k -nearest neighbor (KNN) graph in geographic space using the haversine distance described in Robusto (1957) on latitude/longitude, with each listing connected to its k_{spatial} nearest neighbors. Second, we add a KNN graph in the SBERT embedding space using cosine distance, connecting each listing to its k_{text} most semantically similar descriptions. We then merge and symmetrize these edges so that the final graph encodes both spatial proximity and, when $k_{\text{text}} > 0$, textual similarity.

We use a two-layer GraphSAGE-style message-passing architecture. Starting from $h_i^{(0)} = x_i$, each layer updates node representations by aggregating neighbor information:

$$m_i^{(\ell)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} h_j^{(\ell)}, \quad h_i^{(\ell+1)} = \text{ReLU}(W_{\text{self}}^{(\ell)} h_i^{(\ell)} + W_{\text{neigh}}^{(\ell)} m_i^{(\ell)} + b^{(\ell)}),$$

for $\ell = 0, 1$, where all matrices and biases are trainable. In practice, we implement this using two SAGEConv layers with hidden dimension 128, followed by ReLU, dropout, and a final linear layer producing $\hat{y}_i = w^\top h_i^{(2)} + c$.

Consistent with our geo-aware evaluation, we train the GNN in an *inductive* setting: for each fold, we construct a graph only on the training geo-clusters and learn all parameters by minimizing MSE on log-prices. We then build a separate graph for the held-out clusters and apply the trained model directly to this unseen test graph, forcing generalization to new neighborhoods. The spatial and text KNN sizes (k_{spatial} , k_{text}) are treated as hyperparameters and tuned under the same geo-cluster cross-validation protocol.

5 Results and Discussion

Figure 2 reports the model performance under the three evaluation settings: random 5-fold CV (S1), geo-aware CV without text embeddings (S2), and geo-aware CV with text embeddings (S3). Our main metrics are mean absolute error (MAE) and root mean squared error (RMSE) on the log-price. Working in log-space makes errors approximately correspond to multiplicative errors in price. MAE measures the average absolute deviation between predicted and true log prices, it represents the geometric mean of the multiplicative error factors between predicted and true prices. On the other hand, RMSE penalizes larger errors more heavily due to the square term.

Model	S1		S2		S3	
	MAE	RMSE	MAE	RMSE	MAE	RMSE
KNN (baseline)	0.345	0.474	0.437	0.569	0.514	0.647
Linear Regression	0.356	0.459	0.416	0.519	0.451	0.570
LASSO	0.356	0.459	0.412	0.516	0.389	0.490
Ridge	0.356	0.459	0.412	0.515	0.433	0.547
Random Forest	0.276	0.386	0.347	0.452	0.346	0.448
XGBoost	0.271	0.370	0.353	0.455	0.341	0.439
Simple Neural Network	0.320	0.432	0.426	0.559	0.510	0.657
GNN	/	/	/	/	0.491	0.630

Figure 2: Model Performance Under Random & Geo-Aware Cross-Validation Splits With and Without Text Embeddings

5.1 Cross-validation and hyperparameters

We use two evaluation schemes. As a baseline, we apply standard 5-fold cross-validation (CV) that randomly shuffles listings and implicitly assumes i.i.d. samples. Our main protocol is a geo-aware 5-fold GroupKFold, where folds correspond to disjoint spatial clusters of listings (Figure 1); this setup avoids leakage across neighborhoods and tests generalization to unseen areas.

Hyperparameters are tuned with Optuna under a nested GroupKFold scheme, meaning that inner folds are used exclusively for model selection while outer folds are reserved for final evaluation, ensuring full spatial separation between tuning and testing. For linear models, we search over moderate LASSO/Ridge penalties to trade off variance from high-dimensional one-hot and embedding features against the $\approx 4,400$ training samples. For tree ensembles and the KNN+PCA pipeline, we constrain maximum depth, learning rate, and neighborhood size to limit overfitting on small geo-clusters while still capturing nonlinear effects. The MLP uses three hidden layers with batch normalization, ReLU, dropout, and Adam: this depth is sufficient to model interactions between tabular and text features without making optimization unstable, while tuned widths and dropout provide explicit regularization. For GraphSAGE, we fix a two-layer architecture with 128 hidden units to avoid oversmoothing on this relatively small graph and only tune the spatial and text KNN sizes (k_{spatial} , k_{text}), since the neighborhood radius is the most problem-specific design choice.

5.2 Quantitative Results

Random vs. Geo-aware Splits: On S1 (random splits), all tabular models perform strongly (e.g., XGBoost MAE 0.271, RMSE 0.370). Under the geo-aware S2 setting, errors increase across all models, confirming that random splits overestimate performance because models can exploit spatial autocorrelation rather than truly generalize across regions.

Effect of Description Embeddings: Comparing S2 (no text) with S3 (SBERT), the impact of embeddings varies by model class. Linear models and MLPs degrade with SBERT, consistent with the intuition that hundreds of noisy dimensions can damage distance-based methods (curse of dimensionality). In contrast, regularized models such as LASSO and XGBoost benefit modestly: LASSO sets 83% of text coefficients to zero, and XGBoost selectively splits on informative embedding dimensions. Because XGBoost+SBERT performs best overall, we select it as our **final model**.

GraphSAGE vs. Tabular Baselines: GraphSAGE achieves MAE 0.491 (RMSE 0.630) on S2, outperforming SBERT-augmented KNN and the MLP, highlighting the value of leveraging neighborhood information. While it remains weaker than XGBoost, the GNN provides neural performance and supports inductive generalization to new listings by rebuilding the graph.

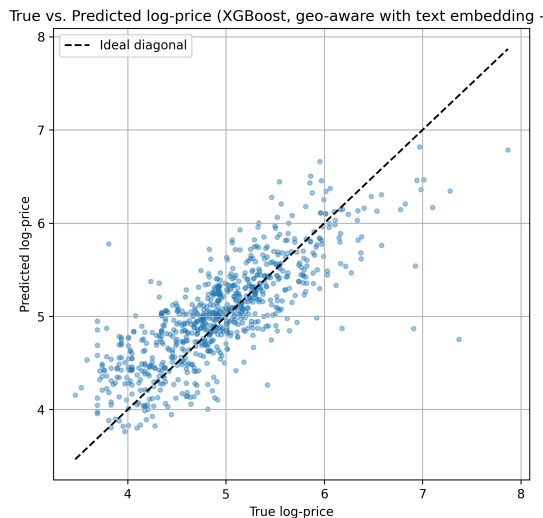
Final Performance for Best Model: Retraining XGBoost+SBERT on the full S2 training set and evaluating it once on the held-out test split yields test MAE 0.312 and RMSE 0.411, closely aligned with cross-validated estimates. Although training metrics remain lower (MAE 0.120), the model generalizes reasonably well despite overfitting moderately.

5.3 Qualitative Analysis:

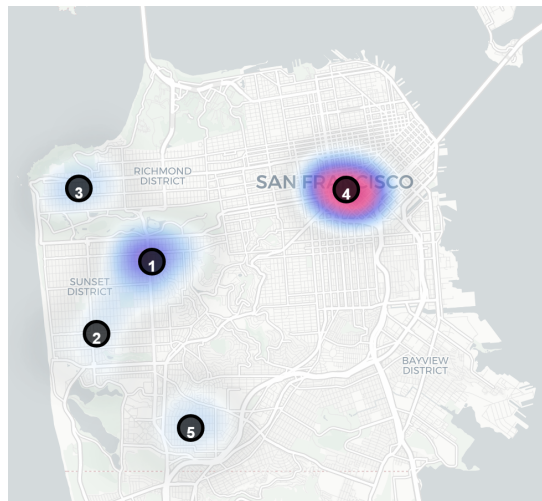
Qualitative inspection supports the quantitative findings. The true-vs.-predicted scatter plot (Figure 3a) shows that XGBoost predictions cluster tightly around the diagonal, while spatial error maps (Figure 3b) reveal a notable hotspot

of large errors in the Northeast of San Francisco. This area contains some of the city’s strongest price variability (luxury towers, tourist areas, low-cost studios), forcing the model into genuine extrapolation rather than local interpolation. Consequently, XGBoost systematically misestimates prices there. Overall, **57%** of listings fall within a $\pm 30\%$ error band on the test set, but coverage varies widely across regions: the more homogeneous coastal cluster 2 reaches **75%**, whereas the heterogeneous downtown cluster 4 reaches only **53%**.

GNN Performance: GraphSAGE is also evaluated on the held-out test split, as it is the central modeling goal of our project. Using the best geo-aware hyperparameters ($k_{\text{spatial}} = 20$, $k_{\text{text}} = 8$), the model attains MAE 0.315 on training and 0.395 on testing. This performance is competitive given the fully inductive setting: the GNN is trained on a graph derived from training clusters and tested on an entirely new graph of unseen nodes and edges. Achieving test MAE ≈ 0.40 indicates that GraphSAGE learns spatial and semantic patterns that transfer across regions. Compared to XGBoost, it shows less overfitting (train MAE close to test MAE), reflecting a smoother inductive bias. Closer inspection of GraphSAGE’s behavior shows that failures often occur where geographic proximity is a poor proxy for price similarity. In cluster 4, listings with very different price levels lie within a small spatial radius, so aggregation mixes incompatible signals. As a result, the GNN underfits sharp local price gradients even when textual similarity edges are included. Additionally, training GraphSAGE required only a few minutes per fold, and graph construction remained the most time-consuming step. This shows that inductive GNNs are computationally lightweight relative to transformer models and tree ensembles.



(a) True vs. predicted log-price for XGBoost



(b) Spatial heatmap of XGBoost error. More intensely pink areas indicate higher prediction error

Figure 3: Qualitative evaluation of XGBoost predictions

6 Conclusion

In this project, we addressed Airbnb price prediction in San Francisco using tabular features, SBERT text embeddings, and spatial information under a geo-aware evaluation protocol. Our best model, XGBoost with SBERT, attains a test MAE of 0.312 in log-price, leveraging nonlinear interactions and regularization to extract a compact, informative subset of textual features while remaining robust to the city’s heterogeneous price landscape.

Our results also underline the importance of realistic spatial evaluation. Under geo-cluster splits, an *inductive* GraphSAGE model achieves test MAE ≈ 0.40 despite never observing the test clusters (or their edges) during training. To the best of our knowledge, this is the first study to combine geo-cluster-based evaluation with an inductive GNN for Airbnb pricing, rather than relying on transductive graph setups with a fixed global graph. This framework shows that graph-based models can competitively transfer spatial and semantic patterns to new regions.

As a next step, we would extend this pipeline to a multi-city, spatio-temporal setting, jointly modeling geography and time (seasonality, demand shocks) on dynamic graphs. This would test the scalability of our geo-cluster + inductive GNN framework and further clarify when graph-based approaches can outperform boosted trees in realistic pricing applications.

7 Contributions

The team worked collaboratively across all stages of the project, including dataset construction, feature engineering, model implementation, hyperparameter tuning, and evaluation under geo-aware validation protocols. While all members participated in discussions, debugging, and report writing, each person took primary ownership of specific components:

- **Ethan:** Led the design of the geo-aware evaluation framework, including the k-means spatial clustering pipeline and the construction of the inductive test environment. Contributed substantially to the modeling design of the GraphSAGE approach, including the definition of inductive graph construction and the overall message-passing architecture. Coordinated the experimental structure and contributed extensively to the discussion of spatial generalization.
- **Owen:** Directed data cleaning, preprocessing, feature engineering, and the full creation of the SBERT-enhanced dataset. Implemented the graph-building pipeline for both spatial and text-based KNNs and carried out the primary GraphSAGE experiments, including training runs, validation monitoring, and ablation variants. Maintained the cross-validation infrastructure and contributed to interpreting GNN behaviour across clusters.
- **Matthieu:** Led the implementation and hyperparameter tuning of all baseline machine learning models, including linear models, tree ensembles, KNN pipelines, and neural networks. Designed and executed large-scale Optuna sweeps across model families, analyzed validation behaviour under geo-aware splits, and contributed extensively to the quantitative and qualitative results analysis.

All members jointly contributed to writing and improving the final report, preparing figures, and synthesizing results. The workload was balanced and coordinated throughout the project.

References

- Inside airbnb. <http://insideairbnb.com/get-the-data/>. Accessed 2024.
- Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD Conference*.
- François Chollet et al. 2015. Keras. <https://keras.io>.
- Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*.
- William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *arXiv preprint arXiv:1706.02216*.
- J. D. Hunter. 2007. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.
- Nikos Kanakaris and Nikos Karacapilidis. 2023. Predicting prices of airbnb listings via graph neural networks and document embeddings: The case of the island of santorini. *Procedia Computer Science*, 219:705–712.
- Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*.
- Wes McKinney. 2010. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 51–56.
- Vinod Nair and Geoffrey E. Hinton. 2010. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning*.
- Optuna Developers. 2019. Optuna: A hyperparameter optimization framework. <https://optuna.org>.
- Fabian Pedregosa et al. 2011. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.
- Liudmila Prokhorenkova, Gleb Gusev, Anna Vorobev, Andrey V. Dorogush, and other contributors. 2018. Catboost. <https://catboost.ai>.
- Python Software Foundation. 2021. Python 3.10. <https://www.python.org/>.
- Python Visualization Community. 2013. Folium: Python library for interactive maps. <https://python-visualization.github.io/folium/>.
- PyTorch Foundation. 2017. Pytorch: An imperative style, high-performance deep learning library. <https://pytorch.org>.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*.
- C. C. Robusto. 1957. The cosine-haversine formula. *The American Mathematical Monthly*, 64(1):38–40.
- Emily Tang and Kunal Sangani. 2015. Neighborhood and price prediction for san francisco airbnb listings. Technical report, Stanford University.
- Stefan Van der Walt, S. Chris Colbert, and Gaël Varoquaux. 2011. The numpy array: A structure for efficient numerical computation. *Computing in Science & Engineering*, 13(2):22–30.
- Dan Wang and Juan L. Nicolau. 2017. Price determinants of sharing economy based accommodation rental: A study of listings from 33 cities on airbnb.com. *International Journal of Hospitality Management*, 62:120–131.
- Michael Waskom. 2021. Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60):3021.
- Yang Yang, Noah J. Mueller, and Robertico R. Croes. 2016. Market accessibility and hotel prices in the caribbean: The moderating effect of quality-signaling factors. *Tourism Management*, 56:40–51.